## Introduction to Object Oriented Programming (OOP)

OOP is defined as a different style of writing code. It is crucial for building real world applications that solve real life problems.

The core idea behind OOP is moving from **generality to specificity**. Instead of relying only on built in data types (like integers, strings, lists, or tuples), OOP allows a programmer to create their **own specific data types** suitable for the application being built.

If coding normally led to code that became confusing and difficult to manage or add new features to after several months, OOP helps in structuring code effectively.

### The Core Problem Solved by OOP

Real life experiences are constantly being converted into software (e.g., banking, booking tickets, photo sharing). When building large, complex software platforms (like a social media site similar to LinkedIn), using only generic, built in data types can result in confusing, unmanageable code. OOP solves this by allowing the creation of data types tailored specifically for the application, resulting in better, optimized, and easier to maintain code.

### Everything is an Object

In Python, **everything is an object**. This includes functions, data structures like lists, and fundamental types like integers. Since everything is an object, every object belongs to a class.

### Core Principles (Pillars) of OOP

OOP is built upon several foundational ideas, which collectively enable this powerful style of coding:

1. **Class**
2. **Object**
3. **Encapsulation**
4. **Polymorphism**
5. **Inheritance**
6. **Abstraction**

---

## Classes and Objects

### Object Definition:

An **Object** is an entity created from a class. An object is an instance of a class.

- **Example:** If `Car` is a class, a specific car (e.g., a "Alto" or a "Toyota") is an object of that class.
- When a variable is created using a data type (which is a class), that variable becomes the object. For instance, if `a = 2` is typed, `a` is an integer object.

**Class Definition (The Blueprint)**

A **Class** is the data type. It serves as the **blueprint** defining how an object will behave. Creating a data type means defining its class or its blueprint. Writing the class requires more effort and code than creating the object.

**Syntax (Conceptual):** A class is defined using the `class` keyword, followed by the class name, and encapsulates Data and Methods.

```
class Class_Name:
  # Data (Attributes/Properties)
  # Methods (Functions/Behavior)
```

**Class Components**

A class comprises two main components:

1. **Data** (also called **Properties** or **Attributes**): Descriptive information about the object.
    - *Example (Car Class):* Number of wheels, color, engine type, brand.
2. **Functions** (also called **Methods** or **Behaviors**): Actions or functionalities the object can perform.
    - *Example (Car Class):* Calculate mileage, calculate speed, open airbag, GPS navigation.

**Naming Conventions (Best Practice)**

When defining a class and its contents, specific Python naming conventions should be followed:

- **Class Names:** Should use **PascalCase** (e.g., `ClassName`), where multiple words are joined and every word starts with a capital letter.
- **Variables (Data) and Functions (Methods):** Should use **snake_case** (e.g., `variable_name` or `function_name`), where words are separated by underscores.

**Creating Objects (Instantiation)**

Objects are created from a class using a specific syntax:

object_name = ClassName()

- *Example:* If `Car` is a class, an object named `my_car` is created as `my_car = Car()`.

- For built-in classes (like `list` or `str`), Python provides a feature called **Object Literal** for easier creation (e.g., `L =`), but internally, the same class instantiation happens.

#### Instance Variables (Instance Attributes)

Instance variables are those variables created inside the constructor (`__init__`).

- **Key Characteristic:** The value of an instance variable is **different for every object** (instance) of the class.
- *Example (ATM Customer):* `pin` and `balance` are instance variables, as every customer object (SBI, HDFC) must have its own unique pin and balance value.

---

## Methods vs. Functions

The distinction between a function and a method is critical in OOP:

| Feature | Function | Method |
|---|---|---|
| **Location** | Defined outside of any class; general purpose. | Defined inside a class. |
| **Accessibility** | Available to be used by any class or object. | Only accessible by an object of that specific class. |
| **Calling Syntax** | The object is passed *inside* the function call (e.g., `len(L)`). | Called using dot notation on the object (e.g., `L.append(4)`). |

- **Best Practice:** When writing OOP code, any function created inside a class should be technically referred to as a **Method**.

---

## Constructors (`__init__`)

#### Definition and Role

The **Constructor** is a special method. In Python, the constructor is specifically named `__init__` (double underscore init double underscore).

- **Role:** The code placed inside the constructor is **automatically executed** the moment an object of that class is created.

**Syntax:**

```
def __init__(self, arg1, arg2, ...):
    # Code that runs immediately upon object creation
    self.data1 = arg1
```

- **Key Detail:** The constructor method name **must** be exactly `__init__` with double underscores (dunder) before and after any typo will prevent it from working.

**Constructors as Magic/Dunder Methods**

`__init__` is categorized as a **Special Method**, **Magic Method**, or **Dunder Method** (due to the double underscores).

- Magic methods are recognized by having double underscores (`__`) before and after the name.
- **Crucial Characteristic:** Magic methods are **not called by the object** explicitly. They are automatically triggered by the Python interpreter upon a specific event. For `__init__`, the trigger event is the creation of the object.

**Philosophy and Use Case**

The primary use of the constructor is for **configuration-related tasks** or actions that must be executed immediately upon the application/object starting, without user input or control.

- Since the user cannot explicitly call the constructor, it is the ideal place for code that cannot be left to user control.
- **Typical Tasks:** Establishing internet connectivity, data connectivity, hardware connection, or setting up GPS.
- In the ATM example, the constructor initializes the instance variables (`pin` and `balance`) to default values and immediately calls the `menu` function to start the user interaction.

---

## The `self` Keyword

**Role and Identity**

The `self` keyword is the first parameter passed to every method inside a class.

- **Identity:** `self` is simply the **object that is currently being worked with**.
- When an object (e.g., `SBI`) calls a method (e.g., `SBI.deposit()`), Python automatically passes that object (`SBI`) as the first input to the method, and this input is received by the `self` parameter.

- Since `self` is the object itself, its memory address (`id(self)`) is the same as the memory address of the object calling the method (`id(SBI)`).

### Necessity of `self`

The `self` parameter is necessary because of a fundamental OOP rule: **a class's method cannot directly access its own class's data or other methods**.

- **The Rule:** Only the class **Object** has the power to access the class's data and methods.
- **The Solution:** To allow one method (e.g., `__init__`) to call another method (e.g., `menu`), or to allow any method to access data (e.g., `self.balance`), it must use the object itself (`self`).
- By using `self.method_name()` or `self.data_name`, the method is accessing components through the object that called it, adhering to the OOP rule.

---

## Encapsulation

**Encapsulation** is the practice of bundling data (attributes) and methods (behaviors) together, and restricting direct access to the data. This is likened to a capsule (powdered medicine inside an outer casing).

### Need for Encapsulation

Leaving data members open and publicly accessible is **not a good practice**.

- If data is left open, a user or junior programmer can directly modify the attribute using statements like `object.attribute = new_value`.
- This direct modification can corrupt the data (e.g., changing `balance` from an integer to a string), causing subsequent code (like the `deposit` function) to crash or malfunction.
- **Purpose:** Encapsulation protects existing code logic by ensuring data integrity.

### Hiding Data (Achieving Privacy)

To prevent direct public access to data, the concept of a **Private Attribute** is used.

- **Python Private Attributes:** Unlike languages like Java which use explicit keywords (e.g., `private`), Python achieves this by placing a **double underscore (`__`)** in front of the variable name.
- *Example:* Changing `pin` to `__pin`.

### Private Members and Name Mangling

When a double underscore is prefixed to an attribute (e.g., `__pin`), Python internally performs **name mangling**.

- The Python interpreter renames the variable to `_ClassName__attributeName` (e.g., `__pin` becomes `_ATM__pin`).
- When accessing the variable inside the class, methods must use the mangled name via `self` (e.g., `self.__pin`).
- **Caveat (Python Philosophy):** Python does not completely hide anything; it is a language "made for adults". A determined programmer who knows the class name and the original attribute name can still access the "private" attribute externally using the mangled name syntax (`object._ClassName__attributeName`).

**Getters and Setters (Controlled Access)**

If the data is hidden, but controlled access is still required, **Getter** and **Setter** methods are implemented.

- **Getter Method (Accessor):** A function whose sole purpose is to **return the value** of a hidden attribute.
  - *Example:* `get_pin()` returns `self.__pin`.
- **Setter Method (Mutator):** A function whose sole purpose is to **change the value** of a hidden attribute.
  - *Example:* `set_pin(new_pin)` changes `self.__pin = new_pin`.
- **Benefit:** Setters allow the programmer to enforce **business logic and rules** before setting a new value (e.g., checking if the new pin is a string, or if the amount is sufficient). The data can only be modified if it passes the logic checks implemented in the setter.

---

## Class Diagrams (UML Notation)

Class diagrams are heavily used in development to represent the structure of classes. A class diagram for the ATM example looks like this:

**ATM (Class Name)**

**Attributes (Data):**

- `__pin` (Private)

- `__balance` (Private)

**Methods:**

- `__init__` (Private)

- `__menu` (Private)

+ `create_pin` (Public)

+ `deposit` (Public)

+ `withdraw` (Public)

+ `check_balance`
(Public)

- **Symbols:**
  - **+ (Plus):** Indicates a **Public** member (accessible outside the class).
  - **- (Minus):** Indicates a **Private** member (intended to be hidden/protected).

---

## Aggregation (Has A Relationship)

**Aggregation** describes a **"Has A" relationship** between classes (e.g., "Customer has an Address").

- **Structure:** One class (the container, or aggregate) holds an object of another class as one of its attributes.
- **Diagram Notation:** Indicated by a diamond shape connecting the aggregating class to the aggregated class.

**Example (Customer and Address)**

Two classes are created: `Address` (containing city, pin_code, state) and `Customer` (containing name, gender, address).

**Address Object Creation:** An `Address` object is instantiated first.

```
class Address:
  def __init__(self, city, pin_code, state):
    self.city = city
    # ...

add_obj = Address("Kot Mithan", "559999", "South Punjab")
```

1.

**Customer Instantiation with Object:** The `Customer` object is created by passing the `Address` object (`add_obj`) into the `Customer` constructor.

```
class Customer:
  def __init__(self, name, gender, address):
    self.name = name
    self.address = address # self.address now holds the Address object
    # ...

customer_k = Customer("A", "M", add_obj)
```

2. **Address Object Creation:** An Address object is instantiated first.

3. **Customer Instantiation with Object:** The Customer object is created by passing the Address object (add_obj) into the Customer constructor

4. **Accessing Nested Data:** Data in the aggregated object is accessed sequentially: `customer_k.address.pin_code`.

- **Behavior:** The aggregating class (`Customer`) can call methods of the aggregated class (`Address`) to perform specific tasks, such as updating the address profile.

---

## Advanced OOP Concepts

### Static Variables (Class Variables)

**Static Variables** (or **Class Variables**) are attributes whose values are **shared across all objects** of the class.

- **Use Case:** They are necessary when all instances need to access the same value (e.g., a bank's IFSC code, a college's name, or a counter).

**Creation:** Static variables are always defined **outside** the constructor (`__init__`) and typically directly under the class definition.

```
class ATM:
  counter = 0 # Static/Class Variable

  def __init__(self):
    # ...
    ATM.counter += 1
    # ...
```

- **Access:** The correct way to access a static variable is using the **Class Name** (e.g., `ATM.counter`). Accessing it using `self.counter` will still reflect the shared value.

**Static Methods**

**Static Methods** are methods that **do not require the object (`self`)** as an input argument.

- **Use Case:** They are primarily used to deal with **static variables**. Since they deal with class-level attributes, they do not need access to instance-specific data provided by `self`.

**Syntax:** They are defined using the `@staticmethod` decorator.

```
class ATM:
  # ... static variable definition ...

  @staticmethod
  def get_counter():
    # Does not take 'self'
    return ATM.counter
```

- **Access:** Static methods can be called directly using the Class Name (e.g., `ATM.get_counter()`), without needing to create an object.

**Pass by Reference (Mutability and Class Objects)**

When an object is passed to a function, it is passed by reference.

- **Identity Preservation:** When an object is passed to a function, the object inside the function receives the exact same reference (memory address/ID) as the original object.
- **Mutability Implication:** Objects created from classes are **mutable**.
  - If a mutable object (like a class object, list, or dictionary) is passed to a function, and that function modifies the object's attributes, the change is permanent and affects the original object outside the function call.
- **Safety Precaution (List/Class Object):** If an original object must be protected from accidental modification when passed to a function, a copy or "cloning" of the object should be passed instead of the original reference.

**Collection of Objects**

Objects created from custom classes can be stored within standard Python data structures (collections) like Lists, Dictionaries, or Tuples.

**Example (List of Objects):**

```
C1 = Customer("Babar", 25)
C2 = Customer("Rizwan", 34)
customer_list = [C1, C2]

for i in customer_list:
```

```
# You can access attributes and methods on the object 'i'
print(i.name)
# i.intro()
```

- Objects stored in collections can be accessed and manipulated using standard iteration and looping techniques.

---

## Inheritance

**Inheritance** is a fundamental OOP concept derived from the real world, where properties and characteristics are passed down.

- **Definition:** It is a technique where one class (the **Child Class** or **Subclass**) acquires (inherits) the properties and behaviors (attributes and methods) of another class (the **Parent Class** or **Superclass**).
- **Relationship:** Inheritance describes an **"Is A" relationship** (e.g., "Smartphone Is A Product," or "Car Is A Vehicle").
- **Directionality:** Inheritance always flows downwards. A Child can inherit from a Parent, but a Parent cannot inherit from a Child.

### Primary Benefit: Code Reusability

The biggest advantage of inheritance is **code reusability**.

- By factoring out common features (like `login` and `register`) into a single Parent class (e.g., `User`), multiple Child classes (e.g., `Student`, `Instructor`) can reuse that code without repetition.
- This adheres to the coding principle **DRY (Don't Repeat Yourself)**.

### What is Inherited

When a Child class inherits from a Parent class, it inherits the following:

1. All **Data Members** (instance variables and static variables).
2. All **Member Functions** (methods).
3. The **Constructor** (`__init__`).

- **Important Exclusion: Private members** (those prefixed with `__`) are **NOT** inherited by the Child class.

### Inheritance Syntax

To make a class inherit from another, the Parent class name is included in parentheses after the Child class name:

```
class Parent:
    # Parent features

class Child(Parent): # Child inherits from Parent
    # Child specific features
```

- A Child object can access all Parent methods.
- A Parent object **cannot** access methods specific to the Child class.

**Calling the Parent Constructor (`super()`)**

There are two scenarios regarding constructor inheritance:

1. **Child Class Lacks Constructor (Implicit Call):** If the Child class does **not** define its own `__init__` method, Python automatically calls the Parent's constructor when the Child object is created.
2. **Child Class Has Constructor (Explicit Call Required):** If the Child class defines its own `__init__`, the Parent's constructor is **NOT** automatically called. If the Parent's initialization logic is still needed (e.g., initializing attributes common to both classes), the Parent's constructor must be explicitly called from inside the Child's constructor.

**The `super()` Keyword**

The `super()` keyword is used to access methods or constructors of the Parent Class from within the Child Class.

- **Invoking Parent Methods:** To call a Parent method (e.g., `buy()`) from a Child method, use `super().method_name()`.
- **Invoking Parent Constructor:** To call the Parent constructor and pass necessary initialization parameters, use `super().__init__(args)`. This is essential for code optimization, allowing the Parent class to handle initialization of common attributes.
- **Best Practice:** When calling the Parent constructor using `super().__init__()`, it should be the **first statement** inside the Child constructor.
- **Limitation:** `super()` can only be used *inside* a class it does not work when called directly on an object outside the class definition.
- **Note:** `super()` can only access Parent methods and constructors; it **cannot** directly access Parent attributes.

**Types of Inheritance**

Python supports four main types of inheritance:

1. **Single-level Inheritance:** One Parent class and one Child class (Child inherits from Parent).
2. **Multi level Inheritance:** A chain of inheritance (Grandparent → Parent → Child). The Child inherits properties from both the Parent and the Grandparent.
3. **Hierarchical Inheritance:** One Parent class has multiple Child classes.
4. **Multiple Inheritance:** A single Child class inherits from two or more Parent classes (e.g., `Child(Parent1, Parent2)`). This is supported in Python but often avoided in other languages like Java.

**Method Resolution Order (MRO)**

In **Multiple Inheritance**, if the same method exists in both Parent classes, a conflict arises. MRO dictates which method is executed based on the order in which the Parent classes were listed in the Child class definition.

- **Priority:** The class listed **first** in the inheritance definition gets priority.
    - *Example:* If `Smartphone(Phone, Product)` is defined, and both `Phone` and `Product` have a `buy()` method, the `Phone`'s `buy()` method will be executed.

---

## Polymorphism

**Polymorphism** means "many forms" (`poly` = multiple, `morphism` = features/forms). It describes the ability of an entity to take on different forms or behaviors.

Polymorphism encompasses three primary concepts: Method Overriding, Operator Overloading, and Method Overloading.

### 1. Method Overriding

Method Overriding occurs when a Child class defines a method that has the **exact same name** as a method in its Parent class.

- **Behavior:** When the Child object calls that method, its *own* method is executed (overriding the Parent's implementation). This is often preferred, as the Child's specific logic (the most recent or relevant implementation) takes precedence.

### 2. Operator Overloading

Operator Overloading is the ability to assign **custom behavior** to standard Python operators (like `+`, `-`, `*`, `>`, etc.).

- **Default Behavior:** An operator has a default behavior (e.g., `+` performs mathematical addition on two numbers).

- **Overloading:** When the operator is used in a context that deviates from its default behavior (e.g., using `+` between two strings results in concatenation, or using `+` between two Fraction objects performs fractional addition), the operator has been overloaded.
- **Mechanism:** Operator overloading is achieved by defining the corresponding **Magic Methods (Dunder Methods)** inside the class.
    - `+` operator uses `__add__(self, other)`.
    - `-` operator uses `__sub__(self, other)`.
    - `*` operator uses `__mul__(self, other)`.
    - `/` operator uses `__truediv__(self, other)`.
    - Magic methods exist for comparison, type conversion, and other arithmetic operations (e.g., `__mod__`, `__pow__`, `__gt__`).

**3. Method Overloading (Python Specific)**

Method Overloading is the concept where a single method name can accept **different numbers or types of arguments** and perform different logic based on the input.

- **Python Behavior:** Unlike languages like Java where true method overloading is supported, Python **does not technically support** method overloading. If two methods share the same name, the second definition will overwrite the first.
- **Python Workaround (Using Default Arguments):** Programmers can achieve the behavior of method overloading by using *default arguments* and checking how many arguments were actually provided within a single method definition.

---

## Dunder Methods (Magic/Special Methods)

Dunder Methods (Magic/Special Methods) are methods defined with double underscores (e.g., `__method__`) that allow classes to integrate with Python's built in behaviors and syntax. They are automatically triggered by the interpreter upon specific conditions.

| Method | Trigger Condition | Purpose |
|---|---|---|
| `__init__` | Object creation (Instantiation) | Initializes instance attributes. |
| `__str__` | When the object is passed to the `print()` function | Defines the human-readable string representation of the object. |
| `__add__` | Using the `+` operator between two objects | Defines custom logic for object addition. |
| `__sub__` | Using the `-` operator between two objects | Defines custom logic for object subtraction. |

| | | |
|---|---|---|
| `__mul__` | Using the `*` operator between two objects | Defines custom logic for object multiplication. |
| `__truediv__` | Using the `/` operator between two objects | Defines custom logic for object division. |

---

## Real world Application Examples

### 1. ATM Software Example (Class, Object, Constructor, Encapsulation)

This example demonstrates creating a software representation of an ATM customer, including data hiding and methods for transactions.

| Concept | Implementation Details |
|---|---|
| **Class** | `class ATM:` |
| **Data (Instance Attributes)** | `__pin` (private) and `__balance` (private). |
| **Constructor** | `def __init__(self):` initializes `self.__pin` (to a blank string) and `self.__balance` (to 0) and calls the `menu` method. |
| **Methods** | `create_pin`, `deposit`, `withdraw`, `check_balance`, and `menu`. |

**Key Code Snippets:**

**Initialization (inside `__init__`):**

```
def __init__(self):
    self.__pin = ""          # Example: Data hiding applied
    self.__balance = 0       # Example: Data hiding applied
    self.menu()
```

**Object Creation & Access:** Creating two separate objects (`SBI`, `HDFC`) demonstrates that they maintain separate, distinct values for `__pin` and `__balance`.

```
SBI = ATM()
HDFC = ATM() # Each object has its own unique instance variables
# Accessing methods:
SBI.deposit()
```

- 
- **Transaction Logic (Withdrawal):** Checks PIN validity, checks if `amount <= self.__balance` before processing withdrawal.

### 2. Fraction Data Type Example (Dunder Methods/Operator Overloading)

This example demonstrates how to create a custom data type that correctly handles arithmetic operations on fractions, using operator overloading.

| Concept | Implementation Details |
|---|---|
| **Class** | `class Fraction:` |
| **Data** | `numerator` and `denominator`. |
| **Instantiation** | Requires two inputs (numerator, denominator). |
| **String Representation** | Uses `__str__(self)` to define how the object appears when printed (e.g., "5/6"). |

```python
def __str__(self):
    return "{}/{}".format(self.numerator, self.denominator)
```